

Clase 3

MÓDULOS

CC1002 Introducción a la
Programación

Descomposición funcional de problemas

- Hemos visto cómo resolver un problema pequeño usando una función
 - Esto permite reutilizar esta solución
- Sin embargo, en problemas más grandes se hace necesario definir muchas funciones auxiliares

CC1002 Introducción a la
Programación

Descomposición funcional de problemas

- Descomposición funcional:
 - Dado un problema, pensar en cómo descomponerlo en problemas más pequeños (subproblemas)
 - Seguir realizando la descomposición hasta que sea posible resolver el subproblema con una función
 - **Una función debe resolver un único subproblema**

CC1002 Introducción a la Programación

Descomposición funcional de problemas

- Pasos a seguir:
 - Siga la receta de diseño para toda función
 - Diseñe funciones auxiliares para cada subproblema o dependencia descubiertos en el enunciado o en los ejemplos de uso
 - Use nombres representativos para variables y funciones

CC1002 Introducción a la Programación

Programas interactivos

- Función **print**: muestra mensaje en la pantalla

```
>>> print('Hola mundo!')
Hola mundo!
```

- Función **input**: devuelve un String con el texto ingresado desde el teclado

```
>>> valor = input('Ingrese un numero: ')
Ingrese un numero: 10
>>> print('El numero ingresado es ' + valor)
El numero ingresado es 10
```

CC1002 Introducción a la
Programación

Programas interactivos

```
>>> numero = input('Ingrese un numero: ')
Ingrese un numero: 10
>>> numero
'10'
>>> int(numero)
10
>>> doble = int(numero) * 2 # doble == 20
>>> nombre = input('Cual es su nombre? ')
Cual es su nombre? Enrique
>>> apellido = input('Cual es su apellido? ')
Cual es su apellido? Jorquera
>>> print('Su nombre es ' + nombre + ' ' + apellido)
Su nombre es Enrique Jorquera
```

CC1002 Introducción a la
Programación

Módulos

- **Módulo:** código correspondiente a un interés común
- **Programación modular:** Separa las funciones de un programa en módulos, los cuales definen una finalidad única y contienen todo lo necesario, código fuente y variables, para cumplirla

CC1002 Introducción a la
Programación

Módulos

- En Python la extensión usada para los módulos (o archivos Python) es `.py`
- Los módulos se cargan con **import**
- Dos módulos importantes predefinidos en Python: **math** y **random**

```
>>> import math  
>>> import random
```

CC1002 Introducción a la
Programación

Módulos

- Módulo **math** (algunas funciones), x , y de tipo **num**, función devuelve **float**

Nombre	Función
<code>math.sqrt(x)</code>	Raíz cuadrada de x ($x > 0$)
<code>math.pow(x, y)</code>	x elevado a y ($x ** y$)
<code>math.exp(x)</code>	e elevado a y ($e = 2.71828...$)
<code>math.log(x)</code>	Logaritmo natural de x
<code>math.sin(x)</code>	Seno de x (x en radianes)
<code>math.cos(x)</code>	Coseno de x (x en radianes)
...	...

CC1002 Introducción a la
Programación

Módulos

- Módulo **random** (algunas funciones), x , y de tipo **int**

Nombre	Función
<code>random.random()</code>	Número aleatorio en $[0, 1)$ (float)
<code>random.randint(x, y)</code>	Número entero aleatorio en $[x, y]$ ($x < y$)

```
>>> import random
>>> random.random()
0.5887452233392187
>>> random.randint(1,10)
10
>>> random.randint(1,10)
2
>>> random.randint(1,10)
2
```

CC1002 Introducción a la
Programación

Módulos

- Otras funciones útiles predefinidas en Python: **abs**, **max**, **min**

```
>>> abs(-5) # valor absoluto
5
>>> max(24, 45, 12) # maximo entre varios numeros
45
>>> min(24, 45, 12, 83) # minimo entre varios numeros
12
```

CC1002 Introducción a la
Programación

Módulos

- Módulos implementados por los usuarios
 - Se guardan en un archivo con extensión `.py`
 - Se cargan con `import`
 - Para evitar tener que colocar el nombre del módulo antes de cada función:

```
>>> import nombreModulo
>>> from nombreModulo import *
```

CC1002 Introducción a la
Programación

Módulos

```
# triangulo.py
import math

# perimetro: num num num -> num
# calcula el perimetro de un triangulo de lados a, b, y c
# ejemplo: perimetro(2, 3, 2) devuelve 7
def perimetro(a,b,c):
    return a + b + c

# Test
assert perimetro(2, 3, 2) == 7

# area: num num num -> float
# calcula el area de un triangulo de lados a,b, y c
# ejemplo: area(3,4,5) devuelve 6
def area(a, b, c):
    semi = perimetro(a, b, c) / 2.0
    area = math.sqrt(semi * (semi - a) * (semi - b) * (semi - c))
    return area

# Test
assert area(3,4,5) == 6
```

CC1002 Introducción a la
Programación

Módulos

```
#ejemplo de uso del modulo triangulo.py
import triangulo
print('Calcular el area y perimetro de un triangulo')
lado1 = float(input('Ingrese el largo del primer lado: '))
lado2 = float(input('Ingrese el largo del segundo lado: '))
lado3 = float(input('Ingrese el largo del tercer lado: '))
print('El perimetro del triangulo es ' + str(triangulo.perimetro(lado1, lado2, lado3)))
print('El area del triangulo es ' + str(triangulo.area(lado1, lado2, lado3)))

#usando from triangulo import *
from triangulo import *
print('Calcular el area y perimetro de un triangulo')
lado1 = float(input('Ingrese el largo del primer lado: '))
lado2 = float(input('Ingrese el largo del segundo lado: '))
lado3 = float(input('Ingrese el largo del tercer lado: '))
print('El perimetro del triangulo es ' + str(perimetro(lado1, lado2, lado3)))
print('El area del triangulo es ' + str(area(lado1, lado2, lado3)))
```

CC1002 Introducción a la
Programación

A considerar

- Las tareas se entregan como módulos

CC1002 Introducción a la
Programación

Para la próxima clase

- Leer el Capítulo 5 del apunte

CC1002 Introducción a la
Programación